# TECHNICAL DOCUMENTATION

System Architecture, Core Algorithms and Security Design




*by Avertix*

Track: AI as a Financial Participant

Sub-track: Cross-border & Student Finance Assistant

Hosted copy: oasis.koreacentral.cloudapp.azure.com

# Contents

Key Notes 3

1. Introduction 5

2. System Architecture 9

3. Core Algorithms 11

4. Security Design 18

5. Evaluation Summary 20

6. Limitations 21

Appendix A. Data Model 23

Appendix B. API Surface 24

Appendix C. Configuration and Reproducibility 25

Appendix D. Repository Layout 26

# Key Notes

Please read these points before using the system.

1. **"Held for your signature" is the correct end state; completion means signed and paid.** Chat drafts a payment but can never approve it; only the student's device-key signature over the exact draft releases it, so a held draft is the designed state, not a failure. To complete one: (1) log in as student and send `pay my rent` to `POST /api/chat`; (2) read `pending_id` and `draft_hash`; (3) press Sign and approve in the web app, or run `python tools/oasis_client.py --base <address> chat "pay my rent" --sign`, which registers a local Ed25519 device key and signs `oasis-approval-v1|role|draft_hash`; (4) the executor re-checks the signature, exact match, nonce and expiry and pays the registry account. Manual signing and examples: `docs/API_REFERENCE.md`, section 4. Task completion here means: draft created from the registry, signed with the student's device key, executor paid; tuition and rent both show PAID. A held draft is a correct end state, not a failure.
2. **Hosted copy and login passcodes (demo roles).** Address: `https://oasis.koreacentral.cloudapp.azure.com`. Passcodes are sent to judges separately by the organisers. Hosted copy: `OASIS_CHAT_LLM=1`, Amazon Nova Pro, so the chat labeller (Section 3.10) uses the live model there: it labels only messages the router cannot place, and never sets a payee, an amount or an approval.
3. **API reference for graders.** `/docs` is off in judge mode, so `docs/API_REFERENCE.md` lists the login, chat, approve (how to sign a draft), state and reset endpoints with example requests and responses.
4. **Source code, tests and deployment.** Repository: `https://github.com/gk-936/oasis`. It holds the README (quick start and results), `tests/` (offline suite, no network), `Dockerfile` and `docker-compose.yml`, and the deployment steps in `docs/deployment-docker.md`; the hosted copy runs from that image on one Azure virtual machine behind Caddy with HTTPS.
5. **The API key has a credit limit.** The key supplied with the project has a limited credit budget. When it is reached, the AI features fall back to the deterministic offline path, so every function keeps working.
6. **Simulated date.** The sandbox clock is fixed at 12 October 2026 (the page banner says so), so bills due in October or November and signing windows that end on 12 October can look past or odd against the real date. Nothing is wrong: the fixed clock is what makes the scripted demo and the evidence logs repeat exactly.
7. **Quickstart.**
    a. Primary: open the hosted copy above, nothing to install, and log in first as **judge** (the 5-minute path starts in the Red team tab); use **student** next for payments and chat, and **parent** to co-sign.
    b. Second option, local install: `scripts\start_judge.ps1` (Windows) or `bash scripts/start_judge.sh` (macOS, Linux). Offline, no model call; it sets judge mode and prints the passcodes.

c. Either way, the 5-minute judge path is in `JUDGE_GUIDE.md`, section “5-minute path”.

# 1. Introduction

Oasis is an AI payment assistant for international students and the families who support them. It is submitted to the track **AI as a Financial Participant**, sub-track **Cross-border & Student Finance Assistant**. This document describes how it is built: the architecture, the algorithms that make decisions, and the security design that limits what the AI can do.

## 1.1 The pain points we are solving

The failures below come from public sources and from informal conversations (see "What people told us" below).

| Problem | Evidence | How Oasis responds |
|---|---|---|
| An impostor landlord asks for rent to be sent to a new account | Shanghai municipal government, anti-fraud guidance for international students (2024). Informal conversations: 2 of the 6 students met a fake landlord (see "What people told us") | Money goes only to the registry account; a changed account restarts the verification ramp |
| A tuition invoice is altered to show a different bank account | KU Leuven invoice-fraud warning; UK university notices | The account on the document is never trusted; a contradiction by the registry is rejected |
| An AI agent is steered by hidden text, or a tool rewrites payment details | Published reports on prompt injection and tool poisoning | Escalate-only AI, canaries, and an executor that enforces the signed intent exactly |
| A large transfer is split to avoid the parent's approval | Recognised structuring pattern | Split-payment detection and sequence anomaly flags hold the payment |
| Sending money through a bank is expensive | World Bank, Remittance Prices Worldwide: bank routes are the costliest provider type. Informal conversations: 3 of the 4 parents struggled with the exchange-rate steps (see "What people told us") | Route planner ranks routes on the amount the recipient receives and reports savings |
| Cost-of-living worry; newcomers invisible to lenders | QS International Student Survey 2023; Statistics Canada 2023 | Forecast and runway triggers; a verified payment record (Trust Pass), not a credit score |

**What people told us.** We held informal, survey-style conversations with 10 people we know: 6 international students and interns abroad (Vietnam, Japan, Germany, Spain and Tanzania) and 4 parents. This is a convenience sample, not a statistical survey, and no finance office was interviewed. Two of the students met a fake landlord while looking for a home and were not scammed: the landlord refused a live viewing, offered only pictures, and when someone was sent to check in person the house did not exist. One student had trouble opening a bank account abroad, which delayed rent and other payments. Three of the four parents could not send money because the exchange-rate steps were involved, and some did not understand them. Three interning students found the difference between exchange rates high when spending. These are individual accounts, not measured frequencies. The notes are in the Evidence folder (`interviews.md`).

## 1.2 The answer

A student abroad pays tuition, rent and living costs across borders, often in a second currency, to payees they cannot easily check, under deadlines. Scams (altered tuition invoices, impostor landlords, look-alike payees), hidden fees and late or short transfers cost real money. An AI assistant could help, but only if it can be given a role in the payment process without being able to move money wrongly.

Oasis answers with one principle: **the AI proposes, deterministic code decides.** The AI reads documents, plans payments, labels chat requests and explains risk. It never approves a payment, signs, holds a credential, changes who is trusted, or writes to the audit log. Every payment passes the same verification, policy, signing and enforcement path whether a person or the AI started it.

## 1.3 What the system covers

| Track need | Oasis feature |
|---|---|
| Tuition payments | Verified-payee tuition flow; altered-invoice defence; invoice intake from text, email and image; duplicate-payment guard |
| Remittances | Route planner (cheapest on-time route against a typical bank route); FlightPath recovery for delayed, rejected or short-credited transfers; peer netting simulation |
| Multi-currency management | Six-currency view and 60-day coverage; FX plan for USD and EUR bills; corridor rules (SAFE quota, LRS, TCS); agentic payment plan with a validator |
| Credit building | Verified payment record and Oasis Trust Pass, with zero-knowledge threshold proofs and a credit-growth plan. A record, not a credit score; records are hybrid post-quantum signed so they remain verifiable long-term |
| Long-lived trust | Post-quantum hybrid signatures (Ed25519 + ML-DSA-65, both required) on every payment intent and proof, and a hybrid post-quantum TLS key exchange (Section 3.13) |

## 1.4 Main features

The main features, named as in the application's tabs. **Start with these five:** Chat, Plan, FlightPath, Break it and Proof; the rest support them.

| Feature | What it does |
|---|---|
| Chat | Talk to Oasis in English or Chinese: ask about bills, rates and runway, estimate a payment, ask for a draft (including a USD or EUR bill), issue a Trust Pass link, track a transfer, check a message for scams or a bank quote for hidden markup. A payment always ends as a draft the student signs. |
| Passport | The student's profile: balances in six currencies, bills, payees and payment history. Plans and checks start from it. |
| Plan | The AI proposes which routes fund which bills in time; the validator checks every amount, deadline and limit, and a plan never skips approval. |
| FlightPath | Follows a sent transfer, says what a delay or short credit means for the bill's deadline, and never resends blindly. |
| Twin | Simulates the rest of the semester and shows the chance of missing each deadline, with mitigations. |
| Netting | Simulation that matches opposite INR and CNY flows at the mid rate to show possible savings. Simulation only. |
| Parent | The parent co-signs larger transfers; the request is shown in English, Tamil, Hindi or Chinese (Chinese is machine-translated; Tamil and Hindi were checked by the team). |
| Demo | Three scripted, repeatable runs: a student's semester, an attack story and bounded autonomy. Each replays to the same audit-chain head. |
| Break it | Judges paste any fraud attempt and watch it run through the real pipeline, or tamper with, replay or forge a signed payment and hand it to the real executor. |
| Red team | Judge-only console for attacking the system. Each judge login gets its own sandbox and attempt log. |
| Proof | Opens a proof pack for a payment and verifies it independently, with a “try tampering” check. |
| Trust Pass | A credential built from verified payments that the student can share in part, with zero-knowledge threshold proofs. Not a credit score. |
| Credit growth | A read-only milestone plan and what-if view over the payment record; it never suggests extra payments. |
| Evidence Passport | One signed, QR-verifiable bundle that shows a payment reached a registry-verified payee. |

| Feature | What it does |
|---|---|
| Invoice intake | Paste text, upload an email or a photo; hidden text is kept as evidence and the account on the document is never trusted. |
| Why-cards | A plain explanation of each hold or refusal, with what is not established and the next step. |
| Corridors and FX | Six corridors with rules for LRS, TCS and the SAFE quota, and foreign-currency plans for USD and EUR bills. |
| Numbers | The committed benchmark results, with their caveats. |
| Post-quantum signing | Every payment intent is signed with two algorithms, one of them post-quantum, and both must verify. |

# 2. System Architecture

## 2.1 Components

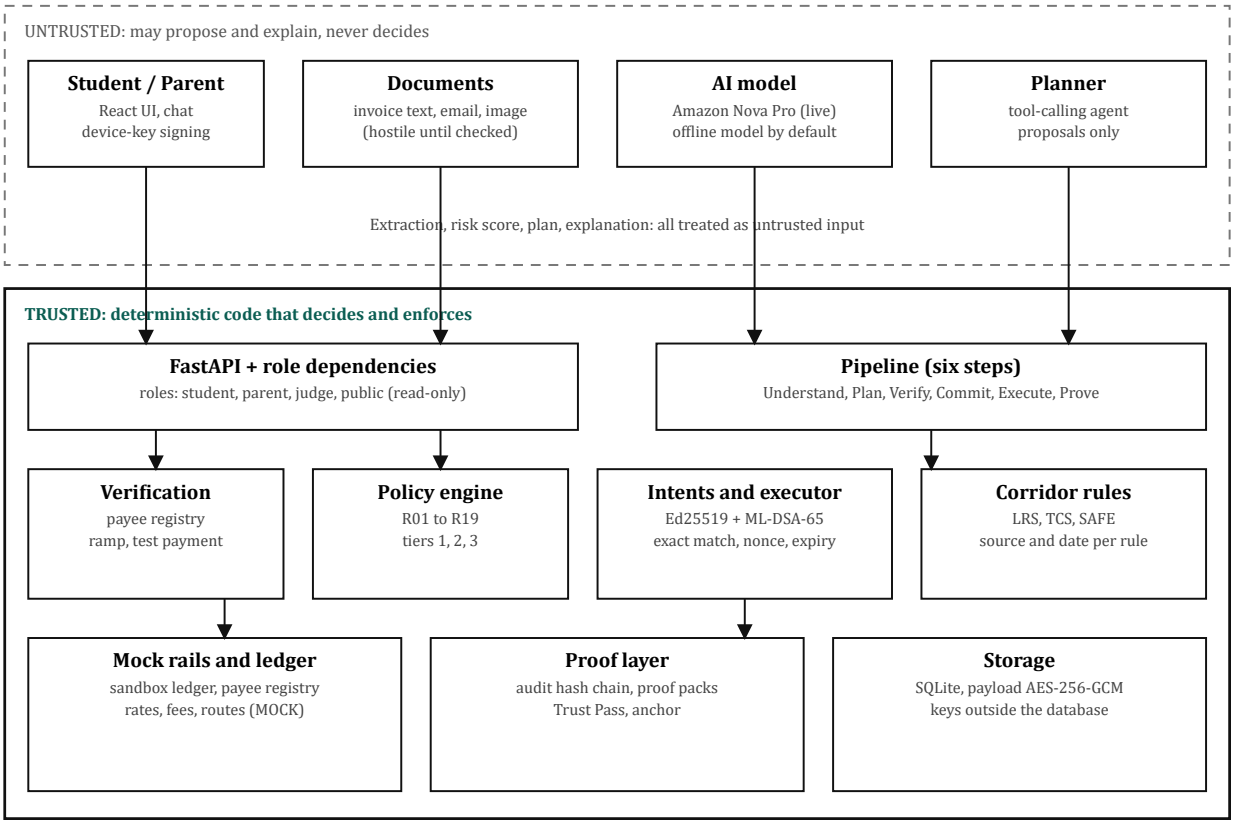



Figure 1. Components and the trust boundary. Arrows show data flowing into trusted code; nothing flows from the AI model to the executor.

| Component | Role | Code |
|---|---|---|
| UI | React 18 and Vite single-page app: chat, Passport, parent view, FlightPath, netting, proof viewer, red-team console. English default, Chinese available (machine-drafted); the parent view also offers Tamil and Hindi (checked by the team). | `frontend/src` |
| API | FastAPI. Every state-changing route declares a role dependency; public routes are read-only. Security headers on every response. | `oasis/api` |
| Agent and planner | Six-step pipeline; tool-calling planner (offline deterministic, or Amazon Nova Pro); chat router and guarded labeller; recovery drafts. | `oasis/agent` |
| Trust layer | Verification ramp, policy engine, signed intents, executor, canaries, corridor rules, audit chain. | `oasis/trust` |

| Component | Role | Code |
|---|---|---|
| Mock rails | Sandbox ledger, payee registry (WeIdentity-style credentials), rates, fees, routes, wallet FX. | `oasis/mock` |
| Corridor rules | Rules as data with source, date and status (INR to CNY and five further corridors). | `oasis/trust/rules` |
| Proof layer | Proof packs, standalone verifier, payment-reliability record, Trust Pass, zero-knowledge thresholds, anchor interface. | `oasis/proof`, `oasis/chain` |
| Storage | SQLite. Audit log written per entry; state saved as a snapshot after each state-changing request; payload columns AES-256-GCM encrypted. | `oasis/storage.py` |

## 2.2 Trust boundary

Documents, screenshots, web pages, tool output and every model output are untrusted. The policy engine, intent builder and signer, executor, verification ramp, registry interface and audit log are deterministic code. Only the executor holds the sandbox payment credential.

| The AI may | The AI may never |
|---|---|
| Read documents and images and extract payment fields | Hold the payment credential or a signing key |
| Score risk (the score can only escalate a decision) | Approve a payment or sign an intent |
| Propose a payment plan by calling read-only tools | Change a payee's verification status or the registry |
| Label a chat message and explain decisions | Write to or edit the audit log |
| Draft a recovery message to the registry's own contact | Choose a payee account (only the registry account is paid) |

## 2.3 The path of one payment

1. **Understand.** The request (typed, pasted, emailed or photographed) is read into fields. A canary scan looks for decoy credentials; the risk scorer adds evidence.
2. **Plan.** Routes are ranked on the amount the recipient receives; the corridor engine adds costs and warnings.
3. **Verify.** The payee is looked up in the registry. The account on the request is never trusted; a contradiction rejects the request without touching the genuine record.

4. **Commit.** Corridor check, then the policy engine decides allow, hold or freeze with a rule ID. Required approvals are device-key signatures over the exact draft hash. An intent is built and signed with two algorithms.
5. **Execute.** The executor verifies both signatures, exact match, nonce and expiry, re-runs the policy, and only then moves money, to the registry account, in the sandbox ledger.
6. **Prove.** A proof pack, audit entry, savings receipt and a plain-language why-card (for holds and refusals) are produced.

### 2.4 Stack and deployment

- **Backend:** Python 3.10+, FastAPI, Pydantic v2, SQLite from the standard library. **Frontend:** React 18 and Vite; the built `frontend/dist` is committed and served by the API, so Node is needed only to rebuild.
- **Model:** an offline deterministic model is the default and the floor for every function. The only live provider is **Amazon Nova Pro on Amazon Bedrock**, through one adapter with a response cache, client-side rate limit, backoff, an on-disk token budget and a deterministic fallback.
- **Hosting:** Docker image (non-root, read-only filesystem, refuses to start without explicitly set passcodes). The submitted copy runs on an Azure virtual machine behind Caddy with a Let's Encrypt certificate. The live key is capped by a lifetime token budget; when it is exhausted or unreachable the system falls back to the deterministic path.
- **Post-quantum:** signatures are hybrid Ed25519 + ML-DSA-65. TLS with hybrid X25519MLKEM768 was tested with Caddy and an OpenSSL 3.5 client (a classical-only client was refused in that test). The deployed site offers X25519MLKEM768 and still accepts classical clients.

### 2.5 Real and simulated

| Real (code that runs as designed) | Simulated (labelled MOCK) |
|---|---|
| Six-step pipeline, policy engine, hybrid signed intents, intent-enforcing executor, verification ramp, canaries, corridor rules engine, hash-chained audit log, proof packs and verifier, plan validator, benchmark harness, Bedrock adapter | Payment rails and balances, route spreads and fees, the payee registry, wallet spending, the clock, simulated humans in benchmarks, anchoring by default (a local hash-linked chain). Mid-market rates are real ECB reference rates when live rates are on, otherwise a snapshot |

## 3. Core Algorithms

Each algorithm is deterministic and reproducible unless noted. Where a model is involved, its output is validated by code before it has any effect.

## 3.1 Payee verification ramp

A payee moves through a fixed set of states. A contradiction by the registry at any point gives REJECTED.

| |
|---|
| **1. UNVERIFIED**: nothing is known about the payee. |
| **2. REGISTRY_CHECKED**: the payee and account match a registry credential. |
| **3. Test payment**: a CNY 1 payment to the registry account succeeds. |
| **4. VERIFIED_LOW**: payments allowed up to CNY 5,000. |
| **5. VERIFIED_FULL**: reached after two successful payments. |

The registry is looked up by payee ID, falling back to an exact name match. If the account on the request differs from the registry credential the request is rejected and the genuine record is untouched. A registry-confirmed account change resets the payee to unverified and re-runs the ramp. Account numbers are normalised for comparison (NFKC, zero-width and format characters removed, spaces and dashes stripped) and then compared exactly, so normalisation can never turn an attacker's number into a registered one.

## 3.2 Policy engine

Rules are evaluated in a fixed order and the first match decides. Every decision records its rule ID. Each decision belongs to one of three tiers:

| Tier | Meaning | Decision |
|---|---|---|
| 1 | The agent may act alone within bounds | allow |
| 2 | Needs explicit human approval | hold, or allow once approved |
| 3 | Never; the payment is stopped | freeze |

| Rule | Decision | Condition |
|---|---|---|
| R01 | freeze (3) | a canary fired |
| R02 | freeze (3) | amount above the hard limit (CNY 200,000 default) |
| R03 | freeze (3) | the registry contradicts the recipient |
| R04 | freeze (3) | risk score at or above 0.85 (escalation only) |
| R04b / R04c | allow (1) / freeze (3) | verification test payment: up to CNY 10 to a registry-confirmed recipient, otherwise freeze |

| Rule | Decision | Condition |
|---|---|---|
| R15 | hold (2) | document demands a currency other than CNY; never paid as if it were CNY |
| R05 | hold (2) | recipient not yet verified; no approval overrides it |
| R18 / R19 | hold (2) / allow (2) | USD or EUR payment: always holds for the student's signature; allowed only with it, a verified recipient and within limits |
| R17 | hold (2) | duplicate: same invoice reference, identical document (30 days) or already-paid obligation; no approver is offered, the student must explicitly choose to pay again |
| R06 | hold (2) | corridor hold: LRS, purpose mismatch, split payments |
| R07 | hold (2) | amount above the recipient's ramp limit |
| R08 | hold (2) | risk score at or above 0.5; no approval releases it |
| R09 / R11 / R12 | approvers / hold / allow (2) | family-support transfers above INR 50,000 need student and parent; missing approvals hold; all approvals present, recipient verified and within limits allows |
| R10 | allow (1) | recurring payment up to CNY 500 to a verified-full recipient |
| R13 / R14 | allow (1) / allow or hold (2) | own-balance conversion at a student-set rate target within CNY 5,000; larger conversions need the student |
| R16 | hold (2) | out-of-band request: a document asking to change approvers or limits, a refund, an unlisted authority's demand, or a message announcing new bank details. No approval path; the student confirms through a registry-verified contact |

## 3.3 Signed intents and the executor

An intent is one exact payment: payee, account, amount, currency, route, nonce, expiry and approvals. Its canonical JSON (sorted keys, no whitespace) is signed with **Ed25519 and ML-DSA-65**; both must verify. The signing interface is algorithm-agnostic (an optional SM2/SM3 signer exists), and only hashes are anchored. The executor pays only if both signatures verify, the payload equals the intent exactly, the intent has not expired, the nonce is unused, and the policy, re-evaluated at execution time, still says allow. Any failure freezes the payment. This is what stops tool-poisoning rewrites after approval, and replays.

### 3.4 Risk scoring

A heuristic scorer weighs urgency language, instructions addressed to the assistant, look-alike or partly matching payee names, an account the registry contradicts, unknown payees, amount mismatch and amounts just under an approval threshold. **Sequence anomaly flags** add evidence from payment history: a burst of payments to several payees in 24 hours (+0.30), a third payment to one payee in 24 hours (+0.20), an amount more than three times earlier ones to that payee (+0.25). An optional trained logistic model and a live-model scorer exist; they combine as the maximum, so **the score can only escalate** (hold at 0.5, freeze at 0.85) and a failed model call falls back to the heuristic. Sanctions screening against a public OFAC snapshot is evidence, never an automatic block or a clearance.

### 3.5 Route planning and corridor engine

The amount delivered to the recipient is:

$$delivered = (send - fixed_fee) \times mid \times (1 - spread) \times (1 - pct_fee)$$

Routes that miss the due date are excluded; the rest are ranked by delivered amount, then speed, then reliability, and savings are reported against a typical bank route. The amount to send is found by binary search on the same formula. Oasis has **six corridors** (INR to CNY, CNY to USD, INR to USD, EUR to USD, INR to EUR, CNY to EUR) on one generic engine. The corridor engine reads rules from YAML data with source, retrieval date and status: LRS (USD 250,000 per remitter per financial year), TCS by purpose, the SAFE quota (USD 50,000 per person per calendar year, counted cumulatively), and split-payment detection (two or more payments to one payee within 72 hours, each just under a threshold). Unconfirmed values are shown as such.

### 3.6 Agentic planner with a deterministic validator

The planner is a tool-using agent whose only outputs are proposals. Its tools are read-only (`read_passport`, `list_obligations`, `get_rates`, `get_routes`, `check_corridor`, `forecast_runway`, plus calculators for route choice and SAFE splits) and `propose_plan`. A plan is a list of convert and pay steps. `validate_plan` simulates it chronologically and returns concrete errors: wrong payee or amount, late payment, unfunded or duplicated obligation, INR going negative, CNY not yet arrived when spent, SAFE or LRS exceeded. Errors go back to the model as tool results; at most four proposals are allowed. If the model cannot produce a valid plan, hits its token cap, or the API fails, the deterministic plan is used and the fallback is recorded. **Trade-offs and questions:** the plan's explanation sets the chosen route against alternatives for the same INR and asks a clarifying question instead of guessing when there is no open bill or no source of funds. Accepting a plan re-validates it; every payment step then enters the normal pipeline, so a plan never bypasses approval.

### 3.7 FX plan and peer netting

Bills owed in USD or EUR are planned by a separate validator and paid under rules R18/R19, with the signature covering the currency; the ledger refuses a transfer whose currency does not match both accounts. Peer netting is a simulation: opposite INR-to-CNY and CNY-to-INR flows are matched greedily (earliest deadline first) and the matched volume settles at the mid rate. Only verified, already-approved payments with at least two days to the deadline are eligible, value is conserved, and a pool of one saves exactly zero. Real netting would need a licensed counterparty.

### 3.8 FlightPath: consequence and recovery

FlightPath follows a payment after it is sent. A journey state machine (prepared, checks complete, submitted, processing, credited, delayed, rejected) is reconciled against the route time and the required amount. The advice names the bill's deadline using plain arithmetic on held facts: days left, how long waiting is safe, whether a replacement could still arrive, and what is not established. **No option ever resends** while the first transfer is unresolved; a new payment needs a rejection and a confirmed refund and then re-enters policy and approval.

### 3.9 Explanations: why-card and rewrite guard

Every hold or refusal produces a why-card with the reason, what the evidence cannot establish, and the next step. These are added after the audit entry is written, so the audit chain is unaffected. In live mode a model may reword a deterministic explanation, but the **rewrite guard** accepts it only if every number is kept, none is added, and the text is plain (no preface, markdown or list); otherwise the deterministic text is used.

### 3.10 Chat router and guarded labeller

The router decides **which existing handler runs** and what words come back. It is deterministic, cannot approve, sign or change a payee, and every payment still ends as a **held draft** that only the student’s device-key signature releases.

| Stage | What it does |
|---|---|
| Security lane | Refuses secrets, bypass, rule changes, authority claims and prompt requests, also when hidden (base64, zero-width, homoglyph); narration and quoted phrases are not refused |
| Scam lane | 12 inducement families (advance fee, safe account, gift cards, …): a warning, never a draft |
| Concept layer | Lexicon and frame parser read the speech act, bill and slots: “take care of the hostel charges” becomes “pay my rent”; statements and worries get an answer plus an offer |

| Stage | What it does |
|---|---|
| Context | Six turns: “and my phone?”, “same again”; a bare “yes” binds only to the last offer |
| Compound requests | Up to four in one message: answered together, first draft made, the rest queued |
| FAQ and fallback | 60+ fixed answers; otherwise a hedge that names the best guess, never a menu for a read-only question |
| Revoke and scorecard | “stop my autopay” is a held draft ended only by signature; the scorecard and mandate card are read from the audit log and shown in the web app as the Agent record panel (Passport for the student, Home for the judge); Stop sends a chat message and ends only on signature |
| Model help | Hosted copy: the labeller is on (a closed-list label for messages the router cannot place, re-checked by every lane above) and the interpreter is off. It never sets a payee, an account or an amount |

**Measured (model off):** **86.0%** on 2,400 generated messages (tuned on), **58.8%** on 1,600 built from words withheld from the lexicon (target 85% not met), **136 of 151** on the frozen set (was 130), **215 of 215** on our attacker catalogue, **CNY 0** moved in every set. Weakest area: wording outside the lexicon.

| | Model off | Model on (interpreter, one run, our own messages) |
|---|---|---|
| Calls and tokens | none | 74 calls, 23,940 tokens in, 2,609 out, about USD 0.03 at list prices |
| Messages the model saw | none | 47 the router could not place (36 DEV messages, 11 attack rows) |
| Pass count, 2,655 messages pooled | 2,327 | 2,340 (+0.49 points, all on DEV: 13 of 36 hedges became passes) |
| Attack rows; money moved | unchanged; CNY 0 | unchanged; CNY 0 |
| Hosted copy | interpreter off | Below the 3-point bar, so the interpreter (`OASIS_CHAT_LLM_MODE=interpret`) is off on the hosted copy; no model-on figure for any other set (`eval/reports/model_on.md`) |

Chat completes these tasks itself, each ending at a draft for a signature, an advice-only answer or a refusal: **fx_bill** (a USD or EUR bill to a fixed MOCK university), **share_record** (Trust Pass, not a credit score), **where**, **plan**, check-in, **scam_check** and **markup_check**. **transfer_issue**, **parent_request** and **quota** are answer-only. Lakh, crore, `1,00,000` and `5k` amounts are read; a number beside “times” is a count.

**Bounded autonomy.** The agent acts alone only inside Tier 1: an own-balance conversion at a student-set rate target up to CNY 5,000 (R13) and a recurring bill of at most CNY 500 to a fully verified registry payee (R10). Everything larger is a held draft that only the student's device-key signature over the exact draft hash releases: a conversion above the auto limit or in another pair (rule C01, `conversion_exec.py`), a standing instruction for a recurring bill and a rate target made through chat (rule C02, `mandate_exec.py`). The cap and the registry payee sit inside the signed hash, the nonce is single use, every automatic run is re-checked by the pipeline and audited, and a bill that rises above its signed cap is held. `GET /api/activity` and the chat question "what did you do for me?" return one timeline of what the agent did alone, proposed, held and refused. A third scripted demo run shows this end to end (24 audit entries, money lost 0, identical chain head on replay). Concept questions (SWIFT, nonce, MOCK, FlightPath, supported currencies) get fixed answers from code, not from a model. On the web form, targets above the CNY 5,000 limit are refused at creation (400: ask in chat and sign the draft), `POST /api/rate-target/approve` releases nothing (409), and only targets within the limit are created without a signature.

### 3.11 Proofs, Trust Pass and credit growth

A proof pack holds the intent, its hash, verification state, costs, policy decision, audit entry and public keys, hybrid-signed. Because the keys in a pack prove nothing alone, verification reports success only when the key fingerprint matches one pinned out of band, and optionally when the anchor exists on an intact chain. The payment-reliability record lists only payments that completed the full pipeline with a verifying proof pack. The Trust Pass is built from it in the SD-JWT style: each entry gets a random salt and the credential signs only digests, so the holder discloses chosen entries or threshold claims ("at least N") and the verifier learns nothing else. Thresholds can be proved with Pedersen commitments and range proofs without revealing counts. It is a record of payment behaviour, **not a credit score**. The credit-growth plan is read-only guidance over that record.

### 3.12 Hash-chained audit log

Each entry stores:

---

*hash = SHA-256( canonical{ i, ts, kind, data, prev } )*

---

The verifier recomputes the chain and reports the first broken link. Payment tokens are single-use, last 30 seconds, are bound to the intent, and never enter the log. The scripted demo uses a fixed seed, mock data, offline model and simulated clock, so each run reproduces the same chain head.

### 3.13 Post-quantum cryptography

Payment records are meant to be checked for years, so Oasis signs them with two independent algorithms and requires both: **Ed25519** (classical) and **ML-DSA-65** (NIST FIPS 204, post-quantum). A signed record stays trustworthy if one algorithm is later broken, which protects against "harvest now,

forge later". Both come from maintained libraries; there is no home-made cryptography.

| Layer | Mechanism |
|---|---|
| Signed intents | The canonical JSON of the intent is signed with a list of algorithms. The executor pays only if every listed signature verifies; removing the ML-DSA-65 signature, or changing one character of the message, is refused. |
| Other signed records | The same hybrid signer signs registry credentials, proof packs, the Trust Pass, the payment-reliability record and the Evidence Passport. |
| Approvals | The approver signs the draft with Ed25519 in the browser; the draft is bound inside the intent that carries the hybrid signature. |
| Transport | TLS 1.3 with the hybrid key exchange X25519MLKEM768, provided by the proxy in front of the application. In the local test a classical-only client was refused. The deployed site offers X25519MLKEM768 and still accepts classical clients (Chrome negotiates the hybrid group; a Windows curl using Schannel connected and got HTTP 200 on). |
| Crypto agility | The signer takes a list of algorithms, so a scheme can be added or retired without changing the executor or the proof format. An optional SM2/SM3 signer (Chinese national standard) was added this way. |

The measurements, the evidence and the limits of this design are in the Security Self-Assessment (Section 7 and the Known-Risk List).

# 4. Security Design

The design assumes that the AI, the documents it reads and every tool it calls can be wrong or hostile, and that safety must hold anyway. The permission tiers, threat model and known-risk list are in the Security Self-Assessment; this section states the design.

## 4.1 Invariants

| Invariant | Enforced by |
|---|---|
| The AI only proposes and explains. It never approves, signs, holds a credential, changes a verification status or writes to the audit log. Its risk score can only escalate. | Pipeline structure; escalate-only combination; tests |
| Money goes only to the registry account, never to an account read from a document. | Verification step and executor payload match |
| The executor enforces the signed intent exactly: signatures, exact match, nonce, expiry, policy re-check. There is no bypass or debug flag. | `executor.py`, `policy.py`, `intents.py` |

| Invariant | Enforced by |
|---|---|
| Every state-changing endpoint uses a role dependency. Public endpoints are read-only and need no secrets. | `api/auth.py`; an API sweep over every route and method |
| Approvals need a device-key signature over the exact draft hash. A role string grants nothing. | Approver registry; browser-held Ed25519 keys |
| A security change needs a test and an entry in the security evidence matrix. | Repository rule |

## 4.2 Defence in depth

- **Untrusted input:** hidden and look-alike characters are normalised and flagged; hidden text in email is kept as evidence; canaries (a fake token, a decoy payee, a decoy account) freeze any payment that touches them.
- **Out-of-band requests (R16):** documents that ask for something Oasis never does from a document are held with no approval path.
- **Access control:** roles student, parent and judge with signed session cookies (HttpOnly, SameSite=Strict); optional passkey second factor; rate and size limits; a red-team console limited to the judge, in a separate sandbox world.
- **Data protection:** AES-256-GCM on payload columns with row-bound data; keys outside the database, optionally passphrase-wrapped; strict response headers (CSP, frame denial, no-referrer).
- **Tamper evidence:** hash-chained audit log; hybrid-signed proof packs verifiable without secrets; intent hashes anchored to a labelled local chain, or to a four-node local FISCO BCOS test chain when enabled.

## 4.3 Degradation behaviour

| Failure | Behaviour |
|---|---|
| Model unreachable, slow (timeout 45 s) or over budget | Deterministic parser, scorer and planner are used; the fallback and its reason are recorded |
| Model returns unusable or unsafe output | Validator or guard rejects it; deterministic text or plan is used |
| Image cannot be read reliably | Low-confidence hold; nothing is paid from a guessed field |
| Live token budget exhausted | Fallback to the offline path for the rest of the run |
| Any executor check fails | The payment is frozen, and the failure is audited |

# 5. Evaluation Summary

Offline results are deterministic and reproducible; live results are from Amazon Nova Pro. Reports are in `eval/reports/`. Caveats are in Section 6.

| Evidence | Result |
|---|---|
| Test suite | 2764 passed, 2 skipped (live FISCO and live PQ-TLS) |
| Visible benchmark: 32 attacks, 12 genuine changes, four layer configurations | Attack block rate 6.2% baseline, 68.8% with verification, 100% with intents and policy; money lost CNY 479,000 baseline, 0 with the full system; genuine completion 100% |
| Hidden attack set: 40 cases (not the router blind set) | 100% blocked and 100% genuine completed after fixes |
| Live extraction, Nova Pro | Model followed an injection in 0 of 6 cases; full system money lost 0; genuine completion 83.3% (a usability cost) |
| Live poisoned invoices | 0 of 128 changed account, amount or payee; 40 of 40 genuine invoices held for signature exactly as offline; 0 payments released without a signature |
| Live planner (deterministic validator) | 56 of 56 plans valid without fallback; 140 of 140 deadlines met in the pooled run |
| Outside-written sets through the live path | 550 attacks and 170 look-alikes, 0 breaches |
| Task completion: chat router, offline, model off | Generated DEV set (2,400 messages, tuned on) 86.0%; generated set from words withheld from the lexicon (1,600, not tuned on) 58.8%, target 85% not met; Router blind set (people who have not seen the router) not yet collected. Frozen set 136 of 151 (was 130); attacker catalogue 215 of 215 (our own); paraphrase probe 75 of 78 (was 47); noise probe 344 of 348. Team held-out set H (never tuned on) 99 of 120. Outside-written sets, same harness, previous to current router: set 3 (ChatGPT) 96 to 97 of 120, set G 138 to 138 of 150, set C 127 to 122 of 150; every attack refused; CNY 0 moved in every set. Those sets were used for earlier fixes, so none is independent |
| Offline planner; executor tamper tests | 8 of 8 plans valid, fees about 83% below a typical bank route; 10 of 10 tampered payments refused |
| Audit chain and replay | Demo reproduces identical chain heads; an edited entry is detected at the first broken link |

# 6. Limitations

## 6.1 What completes and what declines by design

A refusal in the left column is the intended result, not a failed task.

| Request | Oasis result | Why |
|---|---|---|
| Pay rent, tuition or a deposit to a registered payee | Completes: a draft, then paid after the device-key signature | Registry payee, registry account |
| Convert INR to CNY, compare routes, plan the semester, check a scam, share a Trust Pass | Completes: answer, or a draft held for signature | Nothing moves without the signature |
| Pay a new payee, a friend, a classmate or an arbitrary foreign university | Declined: payee not in the registry | Money goes only to registry payees; the verification ramp comes first |
| Pay an account typed in chat or printed on an invoice | Declined or paid to the registry account instead | An account read from a document is never a destination |
| Pay more than the bill, or the same invoice twice | Declined or held (R17) | Overpayment refused; an explicit signed pay-again is needed |
| “Approve it”, “skip approval”, raise limits, reveal passcodes | Refused, audited, CNY 0 moved | Chat can never approve, sign or change rules |
| A USD or EUR bill | Completes only for the two fixed MOCK universities, always held for signature | Foreign-currency payments are never automatic |
| Loans, insurance, visas, investments, opening a bank account | Declined with a list of what Oasis can do | Outside the sandbox’s scope; nothing moved. |
| Above CNY 200,000, a canary or a registry contradiction | Frozen | Tier 3: never |

## 6.2 Limitations

| Limitation | Effect and mitigation |
|---|---|
| Everything financial is simulated | Rails, rates, fees and the registry are MOCK. Savings are not what a real student would save. A production path needs licensing, real rails, external audit and key management. Real bank and payment data was deliberately not used. Moving or reading real money for the INR to CNY corridor needs a licensed payment partner (in India, an authorised dealer bank under FEMA/LRS), and reading real account or student data needs user consent and compliance with India's data-protection law (DPDP Act 2023). A student prototype cannot responsibly hold either. We therefore used mock rails and accounts, and real data only where it is public: ECB reference exchange rates and the OFAC sanctions list. |
| Tests are largely self-written | The attack and chat sets were written by the team or by outside AI systems (given only the pitch and API description, frozen before their first run) and then used to fix failures, so final scores are not out of sample; the first-run row of each report is the only out-of-sample number. Stress tests are white-box. The hidden attack set is partly burned. No external human audit |
| Live evidence is small | One model, one provider, small samples on MOCK data. The chat-label validator rule that removed the one trap label was added after the run and is post hoc. Model calls are not part of the Oasis audit chain: live results are saved report rows, not a provider-side log |
| Some unusual wording is still missed | A clarifying question or a wrong answer-only reply is possible; no money moves (figures in Section 5) |
| Risk scorers are weaker out of sample | They only escalate; the deterministic controls carry the guarantee |
| Image reading can drop a digit | 3 of 30 accepted images in the final live run (33 MOCK images, one run) had an account digit dropped. Payment uses the registry account only and the student confirms the fields; blurry, dark and tiny photos are held by a quality gate before any model call |
| Languages and research | Chinese text is machine-translated; Tamil and Hindi text was checked by the team (the interface is English and Chinese; the parent view is English, Tamil, Hindi and Chinese). User research was a small convenience sample, not a survey (Section 1.1) |

# Appendix A. Data Model

The **Student Financial Passport** (Pydantic v2) holds the student (home and host currency, language, budget), balances per currency, obligations (type, payee, amount, currency, due date, status, recurrence), recipients with a verification record, rules (auto limit, co-approval threshold INR 50,000, hard limit, rate targets), approvers, remitted amounts for LRS and TCS, SAFE usage, funding source, expected income, categorised transactions, the autonomy log and the payment reputation.

| Object | Content |
|---|---|
| Payment intent | payee, account, amount, currency, route, nonce, not_after, approvals, signatures per algorithm |
| Audit entry | i, ts, kind, data, prev, hash |
| Proof pack | intent, hash, verification state, costs, policy decision, audit entry, anchor, public keys, hybrid signature |
| Why-card | reason, uncertainty, next step, rule ID |
| Plan | steps, rationale, tool-call trace, validator feedback, fallback flag, per-step status |
| Ledger entry | sandbox transfer with account, amount, currency, memo |

# Appendix B. API Surface

Generated from `oasis/api/main.py`; interactive documentation is at `/docs` (off in judge mode). Roles: S student, P parent, J judge, A any logged-in role, Public read-only.

| Group | Endpoints | Access |
|---|---|---|
| Session | `/api/health`, `/api/auth/config`, `/api/auth/login\|logout\|me`, passkey routes, `/api/device/register` | Public / A / S, P |
| State | `/api/state`, `/api/reset`, `/api/audit`, `/api/audit/verify` | S, J / S / A |
| Intake and payment | `/api/intake`, `/intake/image`, `/intake/email`, `/api/chat`, `/api/approve`, `/api/pay-again`, `/api/recovery/approve` | S (approve: S, P) |
| Planning | `/api/plan`, `/api/plans`, `/api/plan/{id}/accept\|execute\|consent`, `/api/plan/fx`, `/api/rate-target`, `/api/autonomy/tick` | S (consent: P) |
| Corridors and FX | `/api/corridors`, `/corridors/compare\|check`, `/api/corridor/rules`, `/api/fx/quote\|convert` | A / S |
| FlightPath and netting | `/api/flightpath/*`, `/api/netting/simulate`, `/api/netting/benchmark` | S, J |
| Proofs and credit | `/api/proof/{id}`, `/proof/verify`, `/api/trust/fingerprint`, `/api/reliability/*`, `/api/trust-pass/*`, `/api/credit-growth/*` | Verification: Public; issue: S |
| Screening and parent | `/api/screen`, `/api/parent/pending`, `/api/approvers` | A / P |
| Evaluation and demo | `/api/benchmark`, `/api/demo/*`, `/api/redteam`, `/api/breakit/executor` | S, J (red team: J) |

# Appendix C. Configuration and Reproducibility

## C.1 Configuration

Settings are environment variables loaded from `.env` (never committed).

| Variable | Effect |
|---|---|
| `OASIS_LLM` | `offline` (deterministic, no network) or `bedrock` (Amazon Nova Pro). Offline is forced in tests |
| `OASIS_EXTRACT`, `OASIS_LLM_RISK`, `OASIS_ML_RISK` | Live extraction, live risk scoring and trained-model scoring; all escalate-only |
| `OASIS_RATES` | `live` ECB reference rates (cached, MOCK fallback) or a mock snapshot |
| `OASIS_ANCHOR` | Local simulated chain (default) or `fisco` |
| `OASIS_DB`, `OASIS_DATA_KEY`, `OASIS_KEY_PASSPHRASE` | Database path, data key, passphrase wrapping of keys |
| `OASIS_PASS_*`, `OASIS_JUDGE_MODE`, `OASIS_REQUIRE_PASSKEY` | Role passcodes; judge mode (tighter limits, docs off); passkey second factor |

## C.2 Reproducibility

- Offline mode, fixed seeds, seeded demo keys and nonces, and a simulated clock make the scripted demo repeat exactly; replays reach identical audit-chain heads.
- Benchmarks run offline: `eval.runner --llm offline`, `eval.flightpath_bench`, `eval.netting_bench`, `eval.tasks_bench`. Live runs are checkpointed, token-capped and saved as report rows.
- Test suite: `pytest -q` with an in-memory database and offline model; 2764 passed, 2 skipped.
- The execution evidence folder contains a standard-library verifier that recomputes the audit chains and the headline figures from the logs.

# Appendix D. Repository Layout

```
oasis/
  agent/      pipeline, planner, plan_exec, recovery, autonomy, chat + nlu/router_v2 router (frames,
lexicon, context, scam_lane, faq, scorecard), chat_llm,
              llm + bedrock_llm, risk (heuristic, ml_risk), i18n, localize, duplicates, fx_exec
  api/        FastAPI app, auth, passkeys, rate limits
  trust/      verification, intents, policy, executor, canaries, audit chain, crypto, rules/*.yaml
  planning/   routes, forecast, validator, tools, fxplan, calc
  corridors/  generic corridor engine         flightpath/  journey state machine and advice
  netting/    peer netting simulation         proof/       proof packs, reliability, Trust Pass, zk
  intake/     email, Telegram, IMAP           mock/        ledger, registry, rates, wallet FX
  chain/      anchor interface                market/      live rates
  storage.py  SQLite persistence              demo.py      scripted demo
eval/         benchmarks, scenarios, simulated humans, reports/
frontend/     React + Vite UI (dist committed)
tests/        pytest suite
docs/         technical documentation, security self-assessment, execution evidence notes
```